



Matrix exponentiation



Exponenciação de matriz quadrada

Seja M uma matriz quadrada

- $M^0 = I$ (matriz identidade)
- M^p pode ser calculada em $\log(p)$ por exponenciação rápida (a seguir)



Exponenciação rápida – $O(\log p)$

Por divisão e conquista:

- $a^0 = 1$ e $a^1 = a$
- $a^n = (a^{n/2})^2$, se n for par
- $a^n = (a^{n-1}) \times a$, se n for ímpar

Exemplo:

- $2^9 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \approx O(p)$ multiplications.
- $2^9 = 2^8 \times 2 = (2^4)^2 \times 2 = ((2^2)^2)^2 \times 2 \approx O(\log p)$ multiplications.



Exponenciação rápida – $O(\log p)$

```
int normalExp(int base, int p) {  
    int ans = 1;  
    for (int i = 0; i < p; i++)  
        ans *= base;  
    return ans;  
}
```

código do livro [Comp. Programming 2](#)

```
int fastExp(int base, int p) {  
    if (p == 0) return 1;  
    else if (p == 1) return base;  
    else {  
        int res = fastExp(base, p / 2);  
        res *= res;  
        if (p % 2 == 1) res *= base;  
        return res;  
    }  
}
```

cuidado com **int**!

geralmente precisará de long long e usar aritmética modular



Exponenciação rápida de matriz quadrada – $O(n^3 \log p)$

```
// increase this if needed
#define MAX_N 105

// so that we can return a 2D array
struct Matrix { int mat[MAX_N][MAX_N]; };

// O(n^3)
Matrix matMul(Matrix a, Matrix b) {
    Matrix ans; int i, j, k;
    for (i = 0; i < MAX_N; i++)
        for (j = 0; j < MAX_N; j++)
            for (ans.mat[i][j] = k = 0; k < MAX_N; k++) // if necessary,
                ans.mat[i][j] += a.mat[i][k] * b.mat[k][j]; // do modulo arithmetic here
    return ans;
}
```

```
// O(n^3 log p)
Matrix matPow(Matrix base, int p) {
    Matrix ans; int i, j;
    for (i = 0; i < MAX_N; i++)
        for (j = 0; j < MAX_N; j++)
            ans.mat[i][j] = (i == j); // prepare identity matrix
    while (p) { // iterative version of Divide & Conquer exponentiation
        if (p & 1) ans = matMul(ans, base); // check if p is odd (last bit is on)
        base = matMul(base, base); // square the base
        p >>= 1; // divide p by 2
    }
    return ans;
}
```

código do [Comp. Programming 2](#)



Fibonacci

O programa a seguir usa o fato que

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^p = \begin{bmatrix} F(p+1) & F(p) \\ F(p) & F(p-1) \end{bmatrix}$$

Exemplo:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{11} = \begin{bmatrix} 144 & 89 \\ 89 & 55 \end{bmatrix}$$

Obs.: a dedução da fórmula, a seguir, usa a matriz

$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ e então $F(p+1)$ aparecerá embaixo de $F(p)$.



Fibonacci – $O(\log n)$

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MOD = 1e9 + 7;
7
8  using Matrix = array<array<ll, 2>, 2>;
9
10 Matrix mul(Matrix a, Matrix b) {
11     Matrix res = {{0, 0}, {0, 0}};
12     for (int i = 0; i < 2; i++) {
13         for (int j = 0; j < 2; j++) {
14             for (int k = 0; k < 2; k++) {
15                 res[i][j] += a[i][k] * b[k][j];
16                 res[i][j] %= MOD;
17             }
18         }
19     }
20
21     return res;
22 }
```

```
23
24 int main() {
25     ll n;
26     cin >> n;
27
28     Matrix base = {{1, 0}, {0, 1}};
29     Matrix m = {{1, 1}, {1, 0}};
30
31     for (; n > 0; n /= 2, m = mul(m, m)) {
32         if (n & 1) base = mul(base, m);
33     }
34
35     cout << base[0][1];
36 }
```



Fibonacci

Queremos uma matriz X que gera o número seguinte da série de Fibonacci:

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \end{bmatrix}$$

Veja que a matriz ao lado cumpre esse papel: $X = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$

$$\text{Ex.: } \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} f(5) \\ f(6) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 8 \end{bmatrix} = \begin{bmatrix} 8 \\ 13 \end{bmatrix} = \begin{bmatrix} f(6) \\ f(7) \end{bmatrix}$$

De forma geral:

$$\begin{bmatrix} f(n) \\ f(n+1) \end{bmatrix} = X^n \cdot \begin{bmatrix} f(0) \\ f(1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



Generalização

Série em que o i -ésimo é uma combinação linear dos k anteriores: $a_i = c_1 a_{i-1} + c_2 a_{i-2} + \dots + c_k a_{i-k}$

Queremos uma matriz M que gera o próximo termos da série:

$$\begin{bmatrix} M_{1,1} & M_{1,2} & \dots \\ \vdots & \ddots & \vdots \\ M_{k,1} & \dots & M_{k,k} \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_k \end{bmatrix} = \begin{bmatrix} a_2 \\ \vdots \\ a_{k+1} \end{bmatrix}$$

Note que a_2 até a_k são os mesmos valores, deslocados uma posição.

(basta uma linha com 0's e um valor 1 na posição correspondente)

E o último termo é a combinação linear dos demais.

$$M = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ c_5 & c_4 & c_3 & c_2 & c_1 \end{bmatrix}$$



Generalização

Já vimos a série em que o i -ésimo é uma combinação linear dos k anteriores:

$$a_i = c_1 a_{i-1} + c_2 a_{i-2} + \dots + c_k a_{i-k}$$

$$M = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ c_5 & c_4 & c_3 & c_2 & c_1 \end{bmatrix}$$

E como fazer com seguinte série?

$$a_i = c_1 a_{i-1} + c_2 a_{i-2} + \dots + c_k a_{i-k} + C$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ c_5 & c_4 & c_3 & c_2 & c_1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ C \end{bmatrix} = \begin{bmatrix} a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ C \end{bmatrix}$$



Permutações

Considere um vetor permutação de 1 a N, que indica o mapeamento de uma permutação

Ex.: [3 1 2 4] indica que o 1º elemento vai para a 3ª posição...: [a_1 a_2 a_3 a_4] é mapeado para [a_2 a_3 a_1 a_4]

[5 7 9 1] => [7 9 5 1] => [9 5 7 1] => ...

Qual o resultado após p passos?

Após um passo:

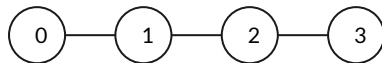
$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ 7 \\ 9 \\ 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 9 \\ 5 \\ 1 \end{bmatrix}$$

Após p passos:

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}^p \begin{bmatrix} 5 \\ 7 \\ 9 \\ 1 \end{bmatrix}$$

Caminhos de tamanho p

Considere o grafo:



Quantos caminhos de 0 até 1 com

- 1 aresta? apenas 1: 0->1
- 2 arestas? nenhum
- 3 arestas? temos 2: 0->1->0->1 e 0->1->2->1
- 4 arestas? nenhum
- 5 arestas? temos 5: 0->1->2->3->2->1 (e outros 4)

Considere sua matriz de adjacência: $M = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

Note que:

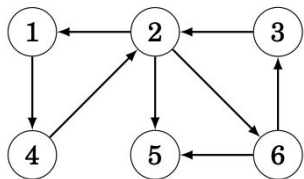
$M^2 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$ informa o número de caminhos entre i e j com 2 arestas.

$M^3 = \begin{bmatrix} 0 & 2 & 0 & 1 \\ 2 & 0 & 3 & 0 \\ 0 & 3 & 0 & 2 \\ 1 & 0 & 2 & 0 \end{bmatrix}$ informa o número de caminhos entre i e j com 3 arestas.

e assim por diante

Caminhos de tamanho p

Considere o grafo:



Sua matriz de adjacência:

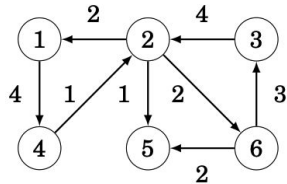
$$V = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

Número de caminhos entre cada par de vértices com exatamente 4 arestas:

$$V^4 = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 \\ 2 & 0 & 0 & 0 & 2 & 2 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Menor caminho com p passos

Considere agora um grafo valorado e sua matriz de adjacência (com distância)



$$V = \begin{bmatrix} \infty & \infty & \infty & 4 & \infty & \infty \\ 2 & \infty & \infty & \infty & 1 & 2 \\ \infty & 4 & \infty & \infty & \infty & \infty \\ \infty & 1 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 3 & \infty & 2 & \infty \end{bmatrix}$$

Considere como "multiplicação" a seguinte fórmula:

$$AB[i,j] = \sum_{k=1}^n A[i,k] \cdot B[k,j] \quad \Rightarrow \quad AB[i,j] = \min_{k=1}^n A[i,k] + B[k,j]$$

(isto é, mínimo em vez de soma e soma em vez de multiplicação)

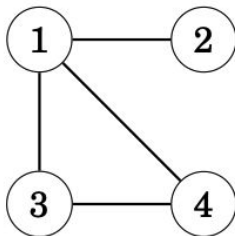
Agora, a exponenciação de matriz traz o caminho mínimo.

Ex.: menor caminho com 4 passos

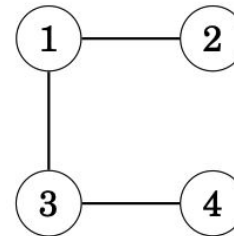
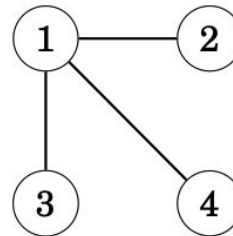
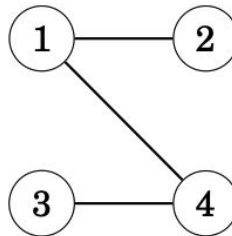
$$V^4 = \begin{bmatrix} \infty & \infty & 10 & 11 & 9 & \infty \\ 9 & \infty & \infty & \infty & 8 & 9 \\ \infty & 11 & \infty & \infty & \infty & \infty \\ \infty & 8 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 12 & 13 & 11 & \infty \end{bmatrix}$$

Número de spanning trees

Quantas spanning trees tem esse grafo?

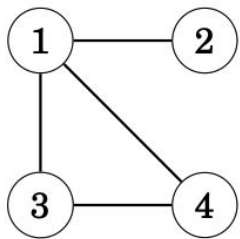


Ele tem 3 spanning trees:



Número de spanning trees (teorema de Kirchhoff)

Matriz Laplaciana
(matriz de graus menos a matriz de adjacência)


$$L = \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 1 & 0 & 0 \\ -1 & 0 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}$$

Número de spanning trees é o determinante da matriz laplaciana retirando-se uma linha e uma coluna

$$\det \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} = 3.$$

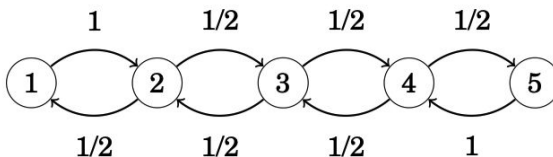
Cadeias de Markov

Uma cadeia de Markov é um processo com estados e transições entre eles. Para cada estado, tem-se as probabilidades de se mover para os demais.

Pode ser representado por um grafo com vértices sendo os estados e arestas as transições.

Como exemplo, considere que você está no 1º andar de um prédio com n andares. Em cada passo, você anda aleatoriamente para o andar de cima ou de baixo (exceto no 1º que só anda para cima e no n que só anda para baixo). Qual a probabilidade de estar no andar k após m passos?

Neste problema cada andar é um estado da cadeia de Markov. Por exemplo, para $n = 5$:





Cadeias de Markov

A distribuição de probabilidade na cadeia de Markov é um vetor $[p_1, p_2, \dots, p_n]$, onde p_k é a probabilidade do estado corrente ser k . ($p_1 + p_2 + \dots + p_n = 1$ sempre).

No exemplo anterior, a distribuição inicial é $[1, 0, 0, 0, 0]$ (sempre começamos no andar 1)

A próxima distribuição é $[0, 1, 0, 0, 0]$ (do andar 1 só podemos ir para o 2)

Depois, podemos subir ou descer um andar, então a próxima distribuição é $[1/2, 0, 1/2, 0, 0]$.

Em seguida, a distribuição é $[0, 3/4, 0, 1/2, 0]$ (do andar 1 ($1/2$) só vamos para o 2 e do andar 3 ($1/2$) podemos ir para o 2 ($1/4$) ou 4 ($1/4$))

Uma forma eficiente de simular o caminho na cadeia de Markov é por Programação Dinâmica:

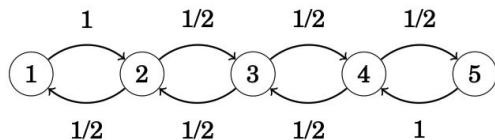
- guardar a distribuição de probabilidade e, em cada estágio, iterar sobre todas as possibilidades que podemos nos mover
- um caminho de m passos pode ser simulado em $O(n^2m)$.

Cadeias de Markov

As transições na cadeia de Markov também podem ser representadas por uma matriz que atualiza a distribuição de probabilidade.

No exemplo anterior, a matriz é:

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix}$$



Ao multiplicarmos essa matriz uma por distribuição de probabilidade, obtemos a próxima distribuição (após a 1 passo)

Ex.: transição da distribuição $[1, 0, 0, 0, 0]$ para $[0, 1, 0, 0, 0]$:

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Usando exponenciação rápida de matriz, podemos obter a distribuição após m passos em $O(n^3 \log m)$.



Fonte

Matrizes: matrix exponentiation, fibonacci, linear recurrences

Grafos e matrizes: counting paths, shortest paths, counting spanning trees

Cadeias de Markov